Hardware-aware Scientific Computing (SoSe 2026) **Exercise sheet 1**

Prof. Dr. Peter Bastian, Nils Friess
IWR, Universität Heidelberg

Submission date: 27. April 2026

Notes:

- You need to show a *significant effort* in the exercises to be admissible for the exam.
- You may submit in groups of two or three people.

Exercise 1 *What hardware am I using?* **(2 points)**

Since we want to do hardware-aware scientific computing, it makes sense to know the hardware of your computer. Find the specifications of your machine and take notes, as you may need them for later exercises. The best way to do this depends on your operating system.

Useful information includes: CPU model, number of cores, clock rate, turbo clock rate, cache sizes (in particular L1/L2 per core), multithreading support, memory bandwidth, and vector instruction sets such as SSE and AVX.

Exercise 2 *Getting the Code* **(0 points)**

This is not really an exercise, but a short note on how to get the example code used below. The code for the lecture is available at `https://parcomp-git.iwr.uni-heidelberg.de/Teaching/hasc-code`.

Clone the repository with:

```
git clone --recursive \
  https://parcomp-git.iwr.uni-heidelberg.de/Teaching/hasc-code.git
```

Read `README.md` and try to build the examples on your machine. Useful git commands include `git clone`, `git status`, `git log`, `git pull`, `git stash`, `git stash pop`, and `git mergetool`.

Exercise 3 *Compiler Flags* **(4 points)**

Look at the examples in the git repository and get familiar with `time_experiment.hh`. Set up experiments to measure the following things:

1. An empty loop that just iterates from 0 to $n-1$ without doing anything.
2. A loop that accumulates something in a global variable.
3. A loop that fills a vector of size n with the integers $0, \cdots, n-1$.

Compile the executable with different compiler flags, for example `-O0`, `-O1`, `-O2`, `-O3`, `-march=native`, `-funroll-loops`, and `-ffast-math`, as well as combinations of them, and compare the runtimes. What can you observe?

Exercise 4 *Parallel* daxpy *Routine* **(4 points)**

In the lecture, you saw a parallel version of the scalar product. Implement the BLAS `daxpy` routine

```
y <- alpha*x + y
```

in a similar way. Use `std::vector<double>` for `x` and `y`, and `double` for `alpha`.